

LECTURE 11

First-Person Fundamentals & New Project Setup

Darkwood Descent — First-Person Survival Horror

Student Guide

LECTURE INFORMATION

Course: Unreal Engine 5 Mastery — From Beginner to Game Developer

Duration: 3 hours

Project: Darkwood Descent (First-Person Survival Horror)

Part 2 of 2 — Lectures 11–15 of 15

Prerequisites: Lectures 1–10 completed, Shadow Ops packaged

Difficulty: Intermediate

Learning Objectives

By the end of this lecture, you will be able to:

1. Understand the design philosophy differences between first-person and third-person games
2. Create a first-person character with smooth camera controls and head bob
3. Build a reusable interaction system using interfaces and line traces
4. Implement an object examination mechanic for inspecting items up close
5. Design a flashlight system with battery drain, recharge, and flicker effects
6. Set up the foundational dark atmosphere for a survival horror environment

RECOMMENDED FREE ASSETS FOR THIS LECTURE

FAB (fab.com):

- Modular Horror Mansion — Complete horror interior kit
- Horror Engine — Sound effects and ambient loops
- Advanced Flashlight — Spotlight system with volumetric light
- Dark Souls Style UI — Dark-themed widget templates

MIXAMO (mixamo.com):

- First-person arm rigs — Search “FPS arms” for hand/arm meshes
- Walking/Running animations — For first-person body awareness

QUIXEL / FAB MEGASCANS:

- Indoor surfaces — Aged wood, cracked plaster, rusty metal
- Debris and clutter — Broken furniture, cobwebs, dust

Part 1: First-Person vs Third-Person Design

1.1 Why First-Person for Horror?

In *Shadow Ops*, we used a third-person camera because tactical shooters benefit from spatial awareness—seeing your character take cover, flank enemies, and navigate open environments. Horror games have the opposite goal: you want the player to feel trapped, vulnerable, and unable to see what is behind them.

First-person perspective achieves this by removing the safety net of a floating camera. The player can only see what the character's eyes can see. Every dark corner requires physically turning to check. Every sound behind you demands you look away from what is in front of you. This is why *Resident Evil 7* switched from third-person to first-person—to heighten the sense of immersion and dread.

REAL GAME EXAMPLE: Resident Evil 7 / Village

Capcom's switch from third-person (RE4–RE6) to first-person (RE7–Village) was a deliberate design choice. First-person made the Baker house feel claustrophobic and Lady Dimitrescu feel towering. The camera is the player's eyes—when something grabs you, the screen shakes and blurs because it is happening to you, not to a character on screen.

1.2 Key Design Differences

Aspect	Third-Person (<i>Shadow Ops</i>)	First-Person (<i>Darkwood Descent</i>)
Camera	Attached to spring arm behind character	Attached directly to character's head bone
Character Visibility	Full body visible at all times	Only hands/arms visible; body hidden
Spatial Awareness	Can see 360° around character	Limited to ~90–110° FOV cone
Immersion	Player observes the character	Player becomes the character
Movement Feel	Responsive, arcade-like	Heavier, more deliberate, weighted
Horror Suitability	Less claustrophobic, more action-oriented	Highly claustrophobic, tension-focused
Interaction	Third-person prompts, visible hands	Direct reach, hands fill screen

KEY CONCEPT: IMMERSION THROUGH LIMITATION

Horror games use limitation as a design tool. Limited visibility (first-person FOV), limited light (flashlight only), limited resources (scarce batteries/ammo), and limited information (no

minimap or enemy markers) all combine to create tension. Every system we build in Darkwood Descent serves this philosophy of purposeful limitation.

Part 2: Darkwood Descent Project Setup

2.1 Creating the New Project

We are starting a brand new project for Darkwood Descent rather than building on Shadow Ops. This gives you experience setting up projects from scratch and keeps the horror game's assets, settings, and lighting configuration completely separate.

Step 1: Open Epic Games Launcher → Unreal Engine tab → Launch UE 5.3+

Step 2: Select the First Person template (not Third Person this time)

Step 3: Set project name to "DarkwoodDescent" (no spaces)

Step 4: Choose Blueprint project type, Maximum Quality, and Raytracing enabled if your GPU supports it

Step 5: Enable Starter Content — we will use some basic materials and meshes for prototyping

Step 6: Click Create Project and wait for the editor to load

2.2 Folder Structure

Create the same professional folder structure we used in Shadow Ops, adapted for horror content:

Folder	Purpose	Example Contents
Blueprints/	All Blueprint assets	BP_HorrorCharacter, BP_Flashlight, BP_Interactive
Materials/	Materials and material instances	M_DarkWood, M_RustyMetal, MI_FlickerLight
Meshes/	Static and skeletal meshes	SM_Hallway, SM_Door, SK_FPSArms
Textures/	Texture files	T_WoodFloor_D, T_Plaster_N, T_Blood_Splatter
Animations/	Animation assets	Anim_Inspect, Anim_FlashlightToggle
Audio/	Sound cues and waves	SC_Footstep_Wood, SC_DoorCreak, SC_Ambient
UI/	Widget Blueprints	WBP_InteractPrompt, WBP_BatteryMeter
Levels/	Map files	L_MainMenu, L_Mansion_Ground, L_Mansion_Basement
VFX/	Niagara and particle systems	NS_DustMotes, NS_FlashlightBeam



PRO TIP

Create a subfolder Blueprints/Interfaces/ right away. We will build multiple Blueprint Interfaces in this project (IInteractable, IDamageable, IHideable), and keeping them organized from the start prevents confusion as the project grows.

2.3 Essential Project Settings for Horror

Before building any gameplay, we need to configure the project for a dark, atmospheric experience:

Setting	Location	Value	Reason
Default Map	Project Settings → Maps	L_Mansion_Ground	Load directly into horror level
Global Illumination	Project Settings → Rendering	Lumen	Dynamic shadows for moving lights
Reflections	Project Settings → Rendering	Lumen	Wet floor reflections, mirror scares
Auto Exposure	Post Process Volume	Disabled	Manual control over darkness levels
Default Game Mode	Project Settings → Maps	GM_DarkwoodDescent	Custom character and controller
Ambient Occlusion	Post Process Volume	Max Intensity 1.0	Deep shadows in corners and crevices

COMMON MISTAKE

Do NOT leave Auto Exposure enabled. Auto Exposure will brighten dark rooms automatically, completely destroying your horror atmosphere. Always disable it in your Post Process Volume and control brightness manually through light placement and post-process exposure settings.

Part 3: Building the First-Person Character

3.1 Character Blueprint Architecture

The first-person character is structurally simpler than our Shadow Ops character. There is no spring arm, no third-person mesh visible to the player, and no complex camera collision. The camera component sits directly on the character and represents the player's eyes.

Component	Type	Purpose
CapsuleComponent	Collision Capsule	Physics body for movement and collision (Height: 96, Radius: 42)
CharacterMovement	Movement Component	Handles walking, crouching, gravity, friction
FirstPersonCamera	Camera Component	Attached to capsule at eye height (~88 units). No spring arm.
FPSArmsMesh	Skeletal Mesh	First-person arms visible to player (optional, Mixamo FPS rig)
FlashlightSpot	Spot Light	Attached to camera — follows player's view direction
InteractionTrace	Scene Component	Origin point for interaction line traces

KEY CONCEPT: CAMERA AS EYES

In third-person, the camera observes the character from behind. In first-person, the camera IS the character's eyes. This means the camera component is attached directly to the capsule at eye height (approximately 88 units up from the capsule center for an average human). There is no spring arm, no camera lag, and no camera collision—the camera moves exactly where the player's head would be.

3.2 Mouse Look and Movement

The Enhanced Input System setup is similar to Shadow Ops, but with important differences for first-person feel:

Step 7: Create an Input Mapping Context: IMC_Horror

Step 8: Create Input Actions: IA_Move (Vector2D), IA_Look (Vector2D), IA_Jump, IA_Crouch, IA_Interact, IA_Flashlight, IA_Sprint

Step 9: Bind IA_Look to Add Controller Yaw/Pitch Input. Enable Use Controller Rotation Yaw on the character (the capsule turns with the mouse, unlike third-person where only the camera turns).

Step 10: Bind `IA_Move` to Add Movement Input using the controller's forward and right vectors. Set Max Walk Speed to 300 (slower than Shadow Ops's 600 for a heavier, more vulnerable feel).

Step 11: Set Ground Friction to 6.0 and Braking Deceleration Walking to 1500 for a weighty stop. Horror characters should not feel nimble.

Movement Speed Comparison

State	Shadow Ops Speed	Darkwood Descent Speed	Feel
Walk	200	150	Cautious, creeping
Normal	375	300	Deliberate, aware
Sprint	600	450	Panicked escape, not athletic
Crouch	175	100	Slow, stealthy, quieter footsteps

3.3 Head Bob System

Head bob simulates the natural up-and-down motion of a person's head while walking. It is essential for first-person immersion—without it, the camera feels like it is floating on rails.

KEY CONCEPT: PROCEDURAL HEAD BOB

We will build the head bob procedurally using sine waves in the Blueprint Tick event, rather than using an animation. This gives us full control over amplitude, frequency, and intensity based on movement speed. Walking produces a gentle sway; sprinting produces an aggressive bounce; crouching produces minimal movement. When the character stops, the bob smoothly settles to zero.

Head Bob Parameters

Parameter	Walk Value	Sprint Value	Description
Vertical Amplitude	2.0 cm	5.0 cm	How far the camera moves up and down
Horizontal Amplitude	1.0 cm	3.0 cm	Side-to-side sway
Vertical Frequency	6.0 Hz	10.0 Hz	Cycles per second (vertical)
Horizontal Frequency	3.0 Hz	5.0 Hz	Cycles per second (horizontal, half of vertical for natural gait)

Parameter	Walk Value	Sprint Value	Description
Interpolation Speed	8.0	8.0	How quickly bob fades in/out when starting/stopping

 PRO TIP

Keep head bob subtle. Excessive camera movement causes motion sickness. Start with the values above and reduce them if testers report discomfort. Many players appreciate an option to disable head bob entirely—consider adding a toggle in your settings menu.

Part 4: Interaction System

4.1 Design Overview

The interaction system is the backbone of a horror game. Players interact with doors, drawers, notes, keys, light switches, hiding spots, and puzzle objects constantly. We need a flexible system that works with any interactable object without hard-coding specific types.

Our approach uses a Blueprint Interface called `BPI_Interactable`. Any actor that implements this interface can be interacted with. The character performs a line trace every frame from the center of the screen; when it hits an actor that implements `BPI_Interactable`, a prompt appears and the player can press E to interact.

REAL GAME EXAMPLE: Alien Isolation / Outlast

Alien: Isolation and Outlast use center-screen interaction systems where the player looks at an object and presses a key. The interaction prompt changes based on the object type: "Open" for doors, "Read" for notes, "Hide" for lockers. We will build the same context-sensitive system using an interface function that returns the prompt text.

4.2 Building the Interface

- Step 12: In Content Browser, right-click → Blueprints → Blueprint Interface. Name it `BPI_Interactable`.
- Step 13: Open it and add a function: `Interact`. Add an input parameter "Caller" of type Actor (the player character).
- Step 14: Add a second function: `GetInteractPrompt`. Add a return value of type Text (the UI prompt like "Open Door" or "Read Note").
- Step 15: Save and close the interface.

4.3 Character Interaction Trace

In `BP_HorrorCharacter`, we add logic on Tick to perform a line trace from the camera forward:

- Step 16: Get the camera's world location and forward vector.
- Step 17: Calculate trace end: $\text{Camera Location} + (\text{Forward Vector} \times 250)$. The 250-unit range means the player must be close to interact—this feels intimate and appropriate for horror.
- Step 18: Perform Line Trace by Channel. On hit, check if the hit actor implements `BPI_Interactable` using "Does Implement Interface."
- Step 19: If yes, call `GetInteractPrompt` (message) to get the text, and show it on the `WBP_InteractPrompt` widget. Store a reference to the currently focused interactable.
- Step 20: When the player presses E (`IA_Interact`), call the `Interact` function (message) on the stored reference, passing `Self` as the caller.

4.4 Example: Interactive Door

Create BP_Door as an Actor that implements BPI_Interactable:

- Step 21: Add a Static Mesh component for the door frame and a separate Static Mesh for the door panel (so the panel can rotate).
- Step 22: Add a Timeline that rotates the door panel from 0° to 90° over 0.8 seconds using an ease-in-out curve.
- Step 23: Implement the Interact function: toggle an IsOpen boolean, play the timeline forward or reversed, and trigger a door creak sound cue.
- Step 24: Implement GetInteractPrompt: return "Open Door" when closed, "Close Door" when open.
- Step 25: Add a boolean "bRequiresKey" and a Name variable "RequiredKeyName." If the door is locked, return "Locked" as the prompt and check the player's inventory for the key in the Interact function.

PRO TIP

Horror doors should creak. Add a slow, ominous creak sound that plays during the Timeline. Vary the sound randomly between 2–3 creak variations so repeated use does not feel repetitive. Resident Evil Village uses 5+ door sound variants per door type.

Part 5: Object Examination Mechanic

5.1 Design Philosophy

The examination system lets the player pick up and rotate objects to inspect them closely. This serves two purposes: delivering story through physical objects (notes, photographs, keys with engravings) and creating vulnerable moments where the player's attention is focused on a small item while danger lurks around them.

REAL GAME EXAMPLE: Outlast / Amnesia

In Outlast, picking up a document fills the screen with readable text while the world continues around you. In Amnesia, you physically pick up objects with the mouse and rotate them. Our system combines both: items appear in front of the camera at a fixed distance, and the player rotates them with the mouse to find clues or read text on different surfaces.

5.2 Implementation Steps

- Step 26: Create a new Blueprint Interface: BPI_Examinable with a function GetExamineTransform (returns a Transform for how the object should appear in front of the camera).
- Step 27: In BP_HorrorCharacter, add a Scene Component called ExamineAnchor positioned 50 units in front of the camera. This is where inspected objects will appear.
- Step 28: When the player examines an object: disable movement input, disable mouse look, attach the object to ExamineAnchor, and enable a special "Examine" input mode where mouse X/Y rotates the object instead of the camera.
- Step 29: Add scroll wheel zoom: Lerp the object between 50 units (default) and 25 units (close-up) from the camera based on scroll input.
- Step 30: When finished (press E again or Escape), return the object to its world position, re-enable movement and camera controls.

COMMON MISTAKE

When disabling player movement during examination, also disable the AI perception system's ability to hear the player. Otherwise, the monster AI (built in Lecture 14) can detect the player during an examination when they have no ability to move or hide. Alternatively, design it so examination can be interrupted—drop the item instantly if the player hears danger.

Part 6: Flashlight System

6.1 Flashlight as Core Mechanic

The flashlight is not just a visual tool—it is the central survival mechanic. It is the player's only reliable light source. It has limited battery. When it flickers and dies, the player is plunged into near-total darkness. This creates constant tension around resource management: do I keep the flashlight on to see, or turn it off to conserve battery?

REAL GAME EXAMPLE: Amnesia: The Dark Descent / Alien Isolation

Amnesia used oil lanterns with limited oil as the primary light mechanic. Alien: Isolation added a flashlight that could attract the Xenomorph—light becomes both safety and danger. In Darkwood Descent, the flashlight drains battery over time, flickers when low, and the monster AI will be attracted to it in Lecture 14.

6.2 Spotlight Component Setup

Property	Value	Reason
Light Type	Spot Light	Directional cone matches real flashlight
Inner Cone Angle	15°	Bright center beam
Outer Cone Angle	35°	Soft falloff at edges
Attenuation Radius	3000 units	~15 meters of useful range
Intensity	8000 lumens	Bright enough to illuminate surfaces clearly
Light Color	Warm white (4500K)	Slightly warm for incandescent flashlight feel
Cast Shadows	Enabled	Critical—shadows from flashlight create horror atmosphere
Volumetric Scattering	0.5	Visible light beam in fog/dust
Attached To	FirstPersonCamera	Follows player's view direction automatically

6.3 Battery System

Step 31: Create variables: MaxBattery (100.0), CurrentBattery (100.0), DrainRate (1.5 per second), bFlashlightOn (false).

Step 32: On Tick (when flashlight is on): subtract $\text{DrainRate} \times \text{DeltaTime}$ from CurrentBattery. Clamp to 0.

Step 33: When CurrentBattery drops below 20%, begin flickering: use a repeating timer that randomly toggles the spotlight visibility every 0.05–0.3 seconds.

Step 34: When CurrentBattery reaches 0, force the flashlight off and prevent turning it on again until batteries are found.

Step 35: Create BP_BatteryPickup: an interactable item that adds 40 battery charge when collected. Place sparingly in levels.

6.4 Flicker Effect Implementation

The flicker effect uses a combination of intensity variation and random toggling to simulate a dying flashlight:

Battery Level	Behavior	Player Experience
100–50%	Steady light, full intensity	Safe, comfortable exploration
50–20%	Occasional single flicker every 10–15s	Growing unease, reminder to conserve
20–10%	Frequent flickers, reduced intensity to 60%	Panic, searching for batteries
10–0%	Constant flickering, intensity drops to 30%	Near-blind, desperate survival
0%	Flashlight dies completely	Total darkness, maximum vulnerability

Part 7: Initial Horror Atmosphere

7.1 First Room Setup

We will build a single horror room as our testing environment. This room will serve as the foundation for all our systems and will be expanded into a full mansion in Lectures 12–15.

- Step 36: Block out a room using BSP geometry: approximately 600×500×350 units (a cramped, oppressive space).
- Step 37: Delete the default Directional Light, Sky Light, and Sky Atmosphere. The room should be completely dark.
- Step 38: Add a single dim Point Light (intensity 0.5, warm amber color, radius 200) on a table in one corner—a practical light source like a candle or lamp.
- Step 39: Place a door using BP_Door. Place a note on the table using a new BP_ExaminableNote. Place a battery pickup on a shelf.
- Step 40: Add an Exponential Height Fog actor with Volumetric Fog enabled. Set density very low (0.005) for subtle atmospheric haze.
- Step 41: Add a Post Process Volume (Infinite Extent). Disable Auto Exposure, set Manual Exposure to 1.0, add subtle Vignette (0.4) and Film Grain (0.05).

PRO TIP

The most important rule for horror lighting: if you can see everything clearly, it is not dark enough. Your flashlight should be the ONLY way to see most of the room. The single practical light source (candle/lamp) should illuminate only a small area and create long, dramatic shadows. Test by turning off your flashlight—if you can still navigate easily, reduce the ambient light further.

Hands-On Exercises

EXERCISE 1: First-Person Character Setup (25 minutes)

Create BP_HorrorCharacter with a Camera component at eye height (88 units).
 Set up the Enhanced Input System with IMC_Horror and all required Input Actions.
 Implement WASD movement with Max Walk Speed 300, sprint at 450, and crouch at 100.
 Build the procedural head bob system using sine waves on Tick.
 Configure the crouch system: capsule height shrinks from 96 to 60, smooth interpolation over 0.2s.
 Test: walk, sprint, crouch, and jump should all feel heavy and deliberate compared to Shadow Ops.

EXERCISE 2: Interaction System and Door (25 minutes)

Create BPI_Interactable with Interact and GetInteractPrompt functions.
 Add the line trace system to BP_HorrorCharacter on Tick (250-unit range from camera center).
 Create WBP_InteractPrompt: a simple centered text widget that shows "Press E to [action]" at the bottom third of the screen.
 Build BP_Door with Timeline rotation, creak sound, and IsOpen toggle.
 Add a bRequiresKey option with a matching key name.
 Test: walk up to the door, verify the prompt appears and disappears at correct range, open/close with E.

EXERCISE 3: Flashlight with Battery Drain (20 minutes)

Add a Spot Light component to BP_HorrorCharacter attached to the camera.
 Configure inner cone 15°, outer cone 35°, intensity 8000, cast shadows enabled.
 Implement the toggle on/off with IA_Flashlight (F key).
 Build the battery drain system: DrainRate 1.5/sec, flicker below 20%, die at 0%.
 Create BP_BatteryPickup that adds 40 charge and implements BPI_Interactable.
 Test: toggle flashlight, watch battery drain, observe flicker behavior, pick up battery to recharge.

EXERCISE 4: Horror Room Assembly (20 minutes)

Block out a single room with BSP geometry (600×500×350 units).
 Delete all default lights. Add one dim practical light source in a corner.

Place your BP_Door at the room entrance, a battery on a shelf, and 2–3 static mesh props (table, chair, bookshelf).

Add Exponential Height Fog with volumetric fog enabled (density 0.005).

Add a Post Process Volume: disable Auto Exposure, set Vignette to 0.4, Film Grain to 0.05.

Test: the room should be almost entirely dark without the flashlight. The flashlight beam should cast visible shadows and scatter in the fog.

Self-Assessment Checklist

Review each item after completing the lecture. Check off every item confidently before moving to Lecture 12.

Topic	Skill	☐
FPS Design	I can explain why first-person perspective enhances horror immersion	
FPS Design	I understand the key differences between FPS and TPS character setup in UE5	
Character	I can create a first-person character with camera at eye height (no spring arm)	
Character	I can implement head bob using procedural sine wave math on Tick	
Movement	I configured heavier, slower movement speeds appropriate for horror	
Movement	Sprint, crouch, and walk all function with correct speed values	
Interaction	I can create a Blueprint Interface with custom functions	
Interaction	I built a line trace interaction system that detects interactable actors	
Interaction	The interact prompt appears/disappears based on trace distance and target	
Examination	I understand the examine mechanic design (anchor point, mouse rotation, zoom)	
Flashlight	The flashlight toggles on/off and drains battery over time	
Flashlight	Flicker effect activates below 20% battery with random toggling	
Atmosphere	I built a dark test room with only practical light sources	
Atmosphere	Volumetric fog and post-processing are configured for horror mood	

Lecture Deliverables

DELIVERABLES BY END OF LECTURE

-  DarkwoodDescent project created with professional folder structure
-  BP_HorrorCharacter with first-person camera, head bob, and heavy movement feel
-  BPI_Interactive interface with Interact and GetInteractPrompt functions
-  Interaction line trace system with WBP_InteractPrompt widget
-  BP_Door with Timeline animation, creak sound, and optional key lock
-  Flashlight system with battery drain, flicker, and BP_BatteryPickup
-  Object examination mechanic (ExamineAnchor, mouse rotation, zoom)
-  One complete horror test room with darkness, fog, and post-processing

Next Lecture Preview

Lecture 12: Inventory, Items & Player Systems

In the next lecture, we will build a grid-based inventory system inspired by Resident Evil, implement health/stamina/sanity player stats, create multiple item types (consumables, key items, documents, weapons), and add a crafting system for combining items. The horror experience deepens as resource management becomes critical to survival.

PREPARATION TASKS FOR LECTURE 12

1. Download grid/slot UI assets from Fab (search “inventory UI kit” or “horror UI”).
2. Prepare 5–10 simple item icons (health potion, battery, key, note, herb) as 128×128 PNG files.
3. Download ambient sound effects: heartbeat, heavy breathing, hallucination whispers (Freesound.org).
4. Review UMG Widget Blueprints from Lecture 8 (Shadow Ops)—we will reuse the same techniques for inventory UI.
5. Sketch a rough inventory layout on paper: 4×6 grid, item tooltip area, use/drop buttons.

WHAT WE'RE BUILDING TOWARD

By end of Lecture 12, your player will manage a Resident Evil–style grid inventory, track health/stamina/sanity with visible UI, read collectible documents that deliver story and clues, and combine herbs and items to craft healing supplies—all while the sanity meter distorts the screen when standing in darkness too long.

